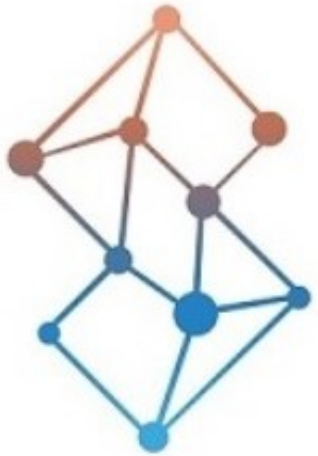
Automated End-to-End Testing

Karel Frajták



STILL

Software Testing IntelLigent Lab

What Is End-to-end Testing?

- End-to-end (E2E) testing is a technique that tests the entire software product from beginning to end to ensure the application flow behaves as expected. It ensures all integrated pieces work together as expected.
- The main purpose of E2E testing is to test from the end user's experience **from one end to another end** by simulating the real user scenario and validating the system under test and its components for integration and data integrity.
- Software systems are complex and interconnected with numerous subsystems. If any of the subsystems fails, the whole software system could crash. This is a major risk and can be avoided by end-to-end testing.

Benefits of E2E testing

- **Ensures the correctness of the application**
- Expands test coverage
- Reduces time to market
- Reduces cost
- Detects bugs
- Prevents regressions

Benefits of E2E testing

Modern systems consist of interacting subsystems. Subsystem failure can affect the whole system.

System risks can be avoided by performing the following:

- Verifying the system flow
- Increasing test coverage areas
- Detecting issues associated with the subsystem

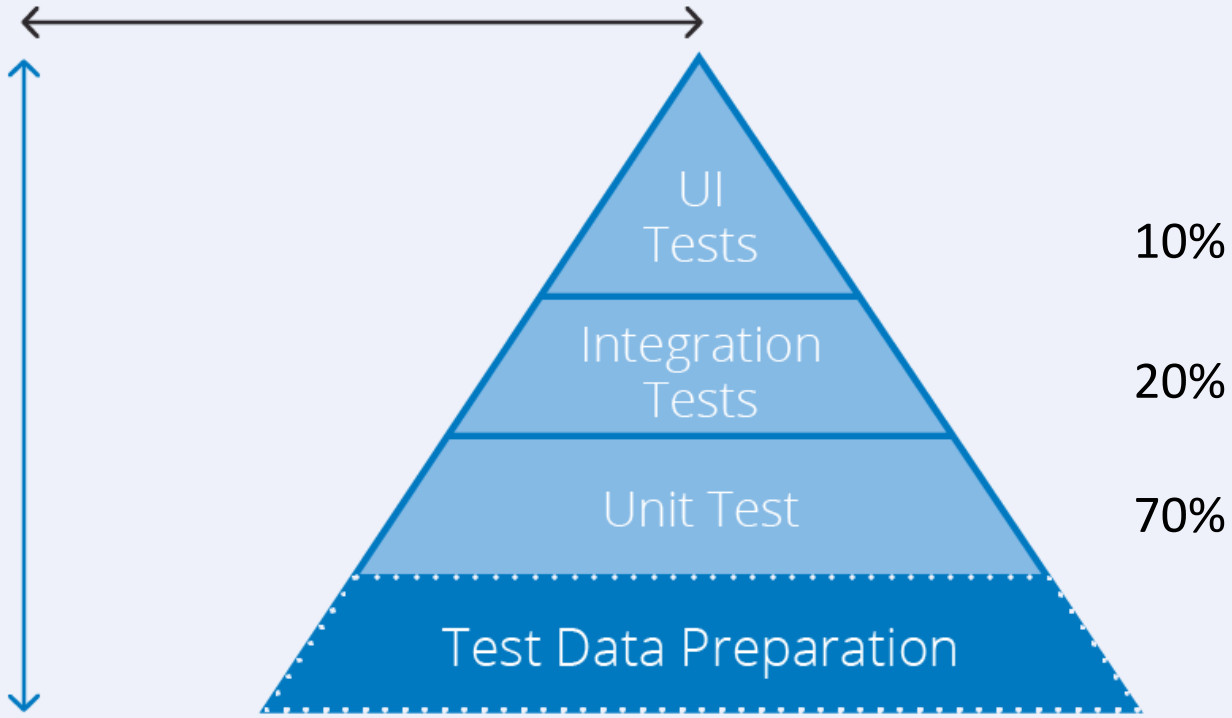
Who is made happy?

- **Developers** benefit from it because most of the testing is offloaded to others.
- **Testers** find it useful because writing tests that verify real-world behavior helps them avoid problems.
- **Managers** like it because user simulation tests let them know how a failing test would impact the user.

E2E Test Automation Horizontal & Vertical Scale

Horizontal E2E (UI & Integration Layer)
Login / Add to Chart / Checkout / Pay

Vertical E2E
Implementing tests on all levels of the pyramid



10%

20%

70%

https://smartbear.com/learn/automated-testing/how-to-perform-end-to-end-testing/
https://martinfowler.com/articles/practical-test-pyramid.html

Horizontal E2E Testing

- Horizontal testing confirms whether a user can navigate through a system, if it works as expected.
- A horizontal end-to-end test would verify each entire workflow or transaction through each individual application from start to finish to ensure that each related process occurs correctly.

Because an entire workflow across multiple applications is being verified, it's vital the test environments for the system and all its sub-systems are set up in advance.

Advantages

- Tests focused on the user perspective
- Prevents issues from reaching production
- Ensures coverage for business logic requirements

Horizontal E2E UI Testing Example

1. Open a browser
2. Navigate to TodoMVC web application site
3. Type “Wear face mask" into the text box on the top
4. Press enter
5. Type “Master E2E testing" into the text box on the top
6. Press enter
7. See that items were added to the list of tasks
8. Once mastered, mark the task as done

Vertical E2E Testing

- Tests happen in a sequential, hierarchal order in layers (see the test pyramid)
- Each component of a system is tested from start to finish ensuring quality.
- Vertical testing is frequently used to test critical components of a complex computing system which do not typically involve users or interfaces.
- A layer without a user interface (UI) would benefit from vertical E2E testing over horizontal.

Advantages

- High code coverage
- Faster test execution
- More focused tests
- Useful for safety critical software

Since tests are focusing on the architecture of a single application, support of a testing or development strategy (BDD, TDD, CI) is required. Ensure that each major stakeholder, such as the developers, tester, product owners, should be involved in the project.

Vertical E2E Testing Example

Evernote example:

1. Test core system granularly with unit tests
2. Test payment processing system
3. Test emailing system
4. Test networks
5. Test API integrations
6. And finally test the UI

Tools

- UI testing
  - Selenium https://www.seleniumhq.org/
  - Cypress.IO https://www.cypress.io/
  - PlayWright https://github.com/microsoft/playwright
  - Robot Framework https://robotframework.org/
  - DevExpress TestCafe https://devexpress.github.io/testcafe/
  - Puppeteer https://github.com/GoogleChrome/puppeteer
- API testing
  - Postman https://www.getpostman.com/
  - SoapUI https://www.soapui.org/
  - Apache JMeter (performance testing as a bonus) https://jmeter.apache.org/

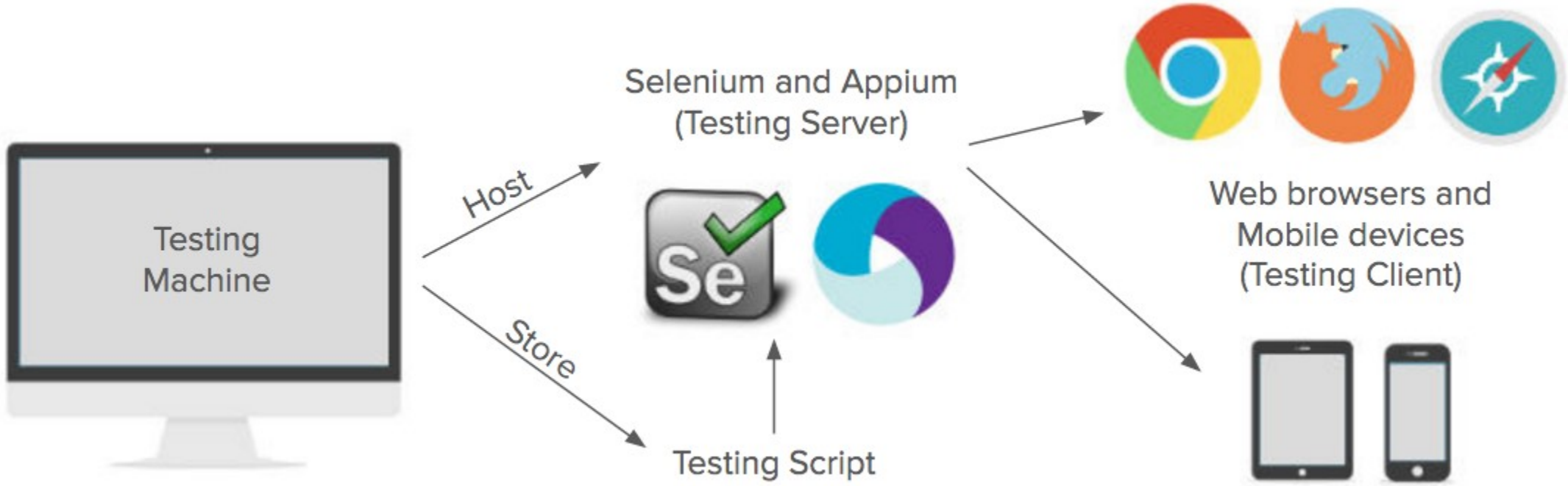
SELENIUM

- Selenium automates browsers!
- What for?
  - E2E Testing
    - You need want to test your web application is behaving correctly? Write code that will “simulate” real end-user actions!
  - Tasks automation
    - You need to perform a task on a website repeatedly and said website does not expose API? Write code that will “simulate” your actions!
  - Data extraction
    - You need to extract tons of data from a website and said website does not expose API? Write code that will “simulate” your actions and get the data!

What Can Selenium Do?

- Fetch page
- Locate UI element
- Get text values
- Handle user input (mouse, keyboard, …)
- Fill in values
- Move between pages or frames
- Handle dialogs
- Navigate
- Manage cookies
- Drag and drop
- Execute script

How it works?


Testing
Machine
Host
Store
Selenium and Appium
(Testing Server)
Se
Testing Script
Web browsers and
Mobile devices
(Testing Client)

How it works?

1. Java client executes Selenium code:
   ```
   WebDriver driver = new ChromeDriver();
   driver.get("http://www.fel.cvut.cz");
   ```
2. POST request is sent to a locally running ChromeDriver with a body
   ```
   POST http://localhost:9515/session
   { "desiredCapabilities": {} }
   ```
3. Server will respond with session id
   ```
   { "sessionId": "e4b3adf2fe9b10fbabd6611d1bb50c93", …}
   ```
4. Navigation to requested page emits another request
   ```
   POST http://localhost:9515/session/e4b3adf2...c93/url
   { "url": "http://www.fel.cvut.cz" }
   ```
5. Driver will navigate the browser to the requested URL

How it works internally?

https://chromium.googlesource.com/chromium/src/+/refs/heads/master/chrome/test/chromedriver/server/http_handler.cc

```
161        CommandMapping(
162            kPost, "session/:sessionId/url",
163            WrapToCommand("Navigate", base::BindRepeating(&ExecuteGet))),
164        CommandMapping(
165            kGet, "session/:sessionId/url",
166            WrapToCommand("GetUrl", base::BindRepeating(&ExecuteGetCurrentUrl))),
167        CommandMapping(
168            kPost, "session/:sessionId/back",
169            WrapToCommand("GoBack", base::BindRepeating(&ExecuteGoBack))),
170        CommandMapping(
171            kPost, "session/:sessionId/forward",
172            WrapToCommand("GoForward", base::BindRepeating(&ExecuteGoForward))),
```

- Chrome WebDriver server
- Accepting WebDriver Protocol Messages
  - JSON Wire Protocol was used before
- Handler for each message that tells the browser what to do

Selenium Drivers

- Browser
  - IE, Chrome, FireFox, Safari
- Mobile
  - Appium, Selendroid, ios-driver, Blackberry
- Desktop
  - WinAppDriver, Winium, QtWebdriver
- Or create your own …
  - See example at https://github.com/kfrajtak/WinAppDriver

SeleniumRC libraries for Java, C#, Python, PHP, Ruby, PERL, JavaScript, …

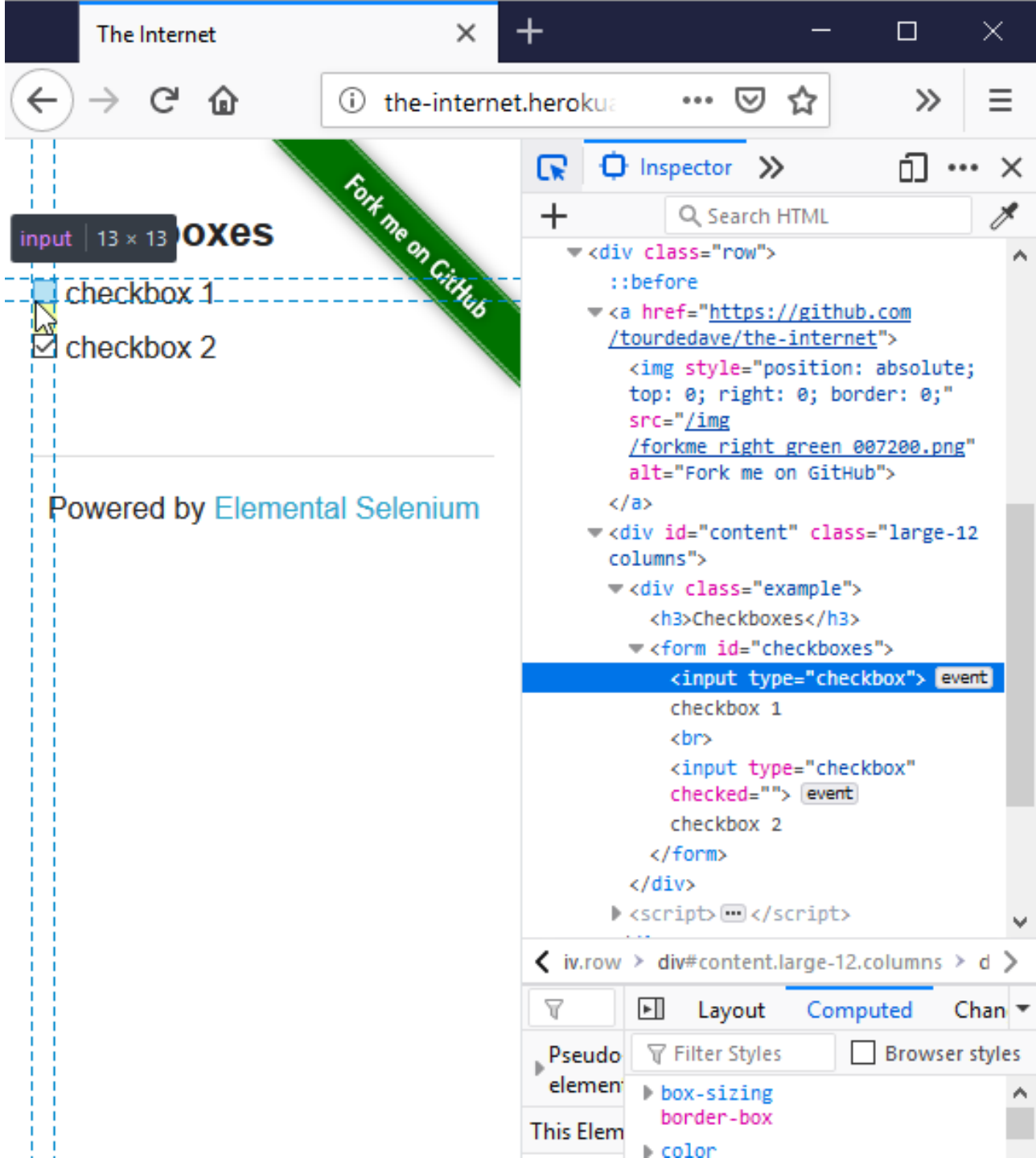
And more https://www.seleniumhq.org/download/

Selenium Locators

Locate element using a locator

- Id
- Name
- XPath
  /html/body/div[2]/div/div/form/input[1]
- Link text
- Partial link text
- Tag name
- Class name
- CSS selector
  #checkboxes > input:nth-child(1)
  html.no-js body div.row div#content.large-12.columns div.example form#checkboxes input

DevTools inspector will be your friend.



Selenium Input Actions

- Simple input actions
- Chainable to perform more complicated interactions
- Touch is supported too (`TouchActions`)

```
Actions builder = new Actions(driver)
    .keyDown(Keys.CONTROL)
    .click(someElement)
    .click(someOtherElement)
    .keyUp(Keys.CONTROL);
    .build();
    .perform();
```

build()	Action
click()	Actions
click(WebElement target)	Actions
clickAndHold()	Actions
clickAndHold(WebElement target)	Actions
contextClick()	Actions
contextClick(WebElement target)	Actions
doubleClick()	Actions
doubleClick(WebElement target)	Actions
dragAndDrop(WebElement source, …	Actions
dragAndDropBy(WebElement source…	Actions
keyDown(CharSequence key)	Actions
keyDown(WebElement target, Char…	Actions
keyUp(CharSequence key)	Actions
keyUp(WebElement target, CharSe…	Actions
moveByOffset(int xOffset, int y…	Actions
moveToElement(WebElement target)	Actions
moveToElement(WebElement target…	Actions
pause(long pause)	Actions
pause(Duration duration)	Actions
perform()	void
release()	Actions
release(WebElement target)	Actions
sendKeys(CharSequence... keys)	Actions
sendKeys(WebElement target, Cha…	Actions
tick(Action action)	Actions
tick(Interaction... actions)	Actions

Selenium Navigation & Options

Navigation

```
driver.navigate().
```

- back()
- forward()
- refresh()
- to(URL url)
- to(String s)

Options

```
driver.manage().
```

timeouts()	Timeouts
addCookie(Cookie cookie)	void
deleteAllCookies()	void
deleteCookie(Cookie cookie)	void
deleteCookieNamed(String s)	void
getCookieNamed(String s)	Cookie
getCookies()	Set<Cookie>
ime()	ImeHandler
logs()	Logs
window()	Window

goTo()

Selenium Wait

A way how tell that XHR has completed.

```
protected boolean waitForJSandJQueryToLoad(WebDriver driver) {

    WebDriverWait wait = new WebDriverWait(driver, timeOutInSeconds: 30);

    // wait for jQuery to load
    ExpectedCondition<Boolean> jQueryLoad = new ExpectedCondition<Boolean>() {
        @Override
        public Boolean apply(WebDriver d) {
            try {
                return ((Long) ((JavascriptExecutor) d).executeScript( s: "return jQuery.active") == 0);
            } catch (Exception e) {
                // no jQuery present
                return true;
            }
        }
    };

    // wait for Javascript to load
    ExpectedCondition<Boolean> jsLoad = new ExpectedCondition<Boolean>() {
        @Override
        public Boolean apply(WebDriver d) {
            return ((JavascriptExecutor) d).executeScript( s: "return document.readyState")
                    .toString().equals("complete");
        }
    };

    return wait.until(jQueryLoad) && wait.until(jsLoad);
}
```

EXAMPLE

This test is probably logging in …

```
@Test
public void step1_loginTest() {
    System.setProperty("webdriver.gecko.driver", "path to geckodriver.exe");
    DesiredCapabilities desiredCapabilities = new DesiredCapabilities();
    FirefoxOptions options = new FirefoxOptions();
    options.merge(desiredCapabilities);
    options.setHeadless(false);
    RemoteWebDriver driver = new FirefoxDriver(options);

    driver.get("http://demo.redmine.org");
    driver.findElement(By.cssSelector("div#account a.login")).click();

    waitForElement(driver, By.id("username"));

    driver.findElement(By.id("username")).sendKeys( ...charSequences: "testuser1358");
    driver.findElement(By.id("password")).sendKeys( ...charSequences: "testit135");
    driver.findElement(By.name("login")).click();
}
```

Driver initialization probably repeated in all tests

Headless mode

What if the structure of the page changes?

Page Object Model

- Design pattern to create **Object Repository** for web UI elements.
- Under this model, for each web page in the application, there should be a corresponding page class.
- This Page class will find the WebElements on that web page and can contain Page methods to interact with those elements.

"Login to a page with these credentials."

vs

"Type 'username' into a text box that can be located using CSS selector '#un'."

POM & Page Object Factory

```java
public class LoginPage {

    @FindBy(how = How.ID, using = "username")
    private WebElement userNameInputElement;

    @FindBy(id = "password")
    private WebElement passwordInputElement;

    @FindBy(name = "login")
    private WebElement loginButtonElement;
    private WebDriver driver;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, page: this);
    }
}
```

- private field for each element on the page
- decorated with @FindBy annotation
  - how & using
  - or just id/name/etc.
- constructor will call PageFactory method to initialize all the fields with elements
  - this way the call to PageFactory will not be omitted

POM & Page Object Factory

- Add public methods for page interaction and/or verification
- Goal of the methods is to hide all Selenium related code that complicates test authoring and understanding and increases future maintenance effort

```java
public static LoginPage goTo(String baseUrl, WebDriver driver) {
    driver.get(baseUrl);
    driver.findElement(By.cssSelector("div#account a.login")).click()
    LoginPage loginPage = new LoginPage(driver);
    loginPage.waitFor();
    return loginPage;
}

public LoginPage enterUserName(String userName) {
    userNameInputElement.clear();
    userNameInputElement.sendKeys(userName);
    return this;
}

public LoginPage enterPassword(String password) {
    passwordInputElement.clear();
    passwordInputElement.sendKeys(password);
    return this;
}

public void login() {
    loginButtonElement.click();
}

public void login(String userName, String password)
{
    enterUserName(userName);
    enterPassword(password);
    login();
}
```

Room for improvement

Page Object Model

Before

```java
@Test
public void step1_loginTest() {
    System.setProperty("webdriver.gecko.driver", "path to geckodriver.exe");
    DesiredCapabilities desiredCapabilities = new DesiredCapabilities();
    FirefoxOptions options = new FirefoxOptions();
    options.merge(desiredCapabilities);
    options.setHeadless(false);
    RemoteWebDriver driver = new FirefoxDriver(options);

    driver.get("http://demo.redmine.org");
    driver.findElement(By.cssSelector("div#account a.login")).click();

    waitForElement(driver, By.id("username"));

    driver.findElement(By.id("username")).sendKeys( ...charSequences: "testuser1358");
    driver.findElement(By.id("password")).sendKeys( ...charSequences: "testit135");
    driver.findElement(By.name("login")).click();
}
```

After

```java
@Test
public void step1_loginTest() throws MalformedURLException {




    RemoteWebDriver driver = getDriver();

    LoginPage loginPage = LoginPage.goTo( baseUrl: "http://demo.redmine.org", driver);




    loginPage.enterUserName("testuser1358")
            .enterPassword("testit135")
            .login();
}
```

```java
@Test
public void step1_loginTest() throws MalformedURLException {
    RemoteWebDriver driver = getDriver();
    LoginPage loginPage = LoginPage.goTo( baseUrl: "http://demo.redmine.org", driver);
    loginPage.login( userName: "testuser1358",  password: "testit135");
}
```

Page Object Model

Advantages

+ Selenium related code is hidden in the page class
+ Functional encapsulation
+ Code reuse
+ Reduced copy&paste style of programming
+ Increased test readability
  + Shorter tests
  + Focused tests
  + Test contains more application logic
+ Low maintenance
  + Changes made to page structure are reflected only in related page object

Disadvantages

- High setup time & effort (YAGNI)
- Application specific
- Code smells
  - Large class – lots of fields and methods for complicated web pages
- Violation of **SO**LID principles, namely
  - **S**RP (Single Responsibility)
  - **O**CP (Open Closed)
- Not entirely story focused
  - Focus is on implementation

Create POMs for each page

Class is doing a lot of things.

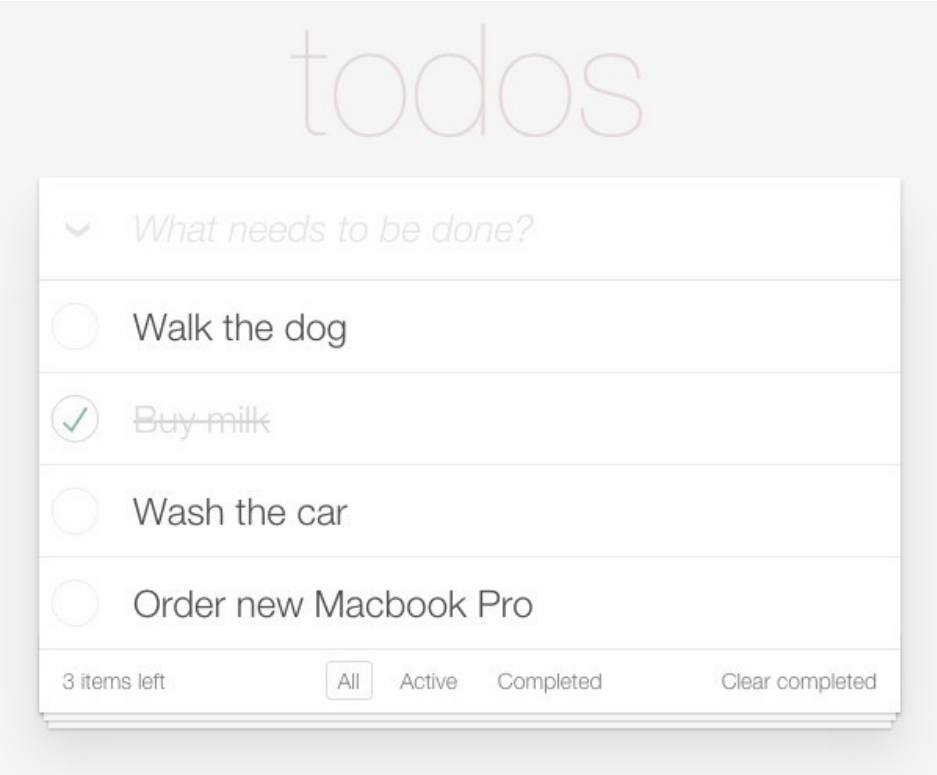
"…you extend the behavior of such modules by adding new code, not by changing old code that already works."

Example

https://ideas.riverglide.com/page-objects-refactored-12ec3541990

A very simple Todos page

http://todomvc.com/



- (Some) Responsibilities
  - newTodo
  - todoItems
  - todosRemaining
  - toggleAllButton
  - clearCompletedButton
  - filterTodos
  - addTodoItem
  - addTodoItems
  - toggleAllCompleted
  - filterItems

How to find elements

How a user completes given task

Another perspective

Different perspective: Each test scenario becomes a narrative describing the tasks and how a story is expected to play out for a given goal.

Behaviors are the primary concern of the E2E tests; the implementation secondary.

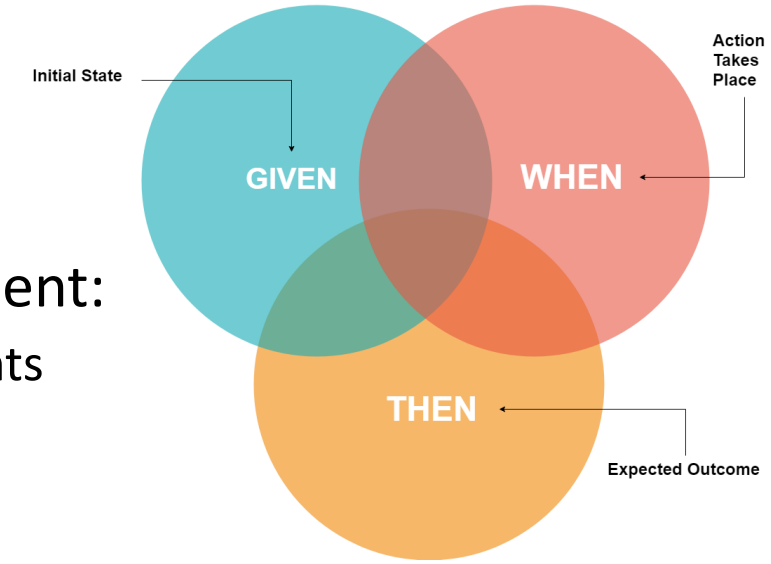
This is especially true if you're employing BDD and writing acceptance tests where the user's ability to complete a goal is more relevant than the eventual solution.

- Roles - Who is this for?
- Goals - Why are they here and what outcome do they hope for?
- Tasks - What will they need to do to achieve these goals?
- Actions - How they complete each task through specific interactions?

New perspective

By starting from this point of view, this changes the way the domain is perceived and therefore how it should be modelled. This thinking **takes testers away from pages to actors with the abilities to play a given role.**

Behavior Driven Development



- BDD is a set of practices that aim to reduce some common wasteful activities in software development:
  - Rework caused by misunderstood or vague requirements
  - Technical debt caused by reluctance to refactor code
  - Slow feedback cycles caused by silos and hand-overs
- BDD aims to narrow the communication gaps between team members, foster better understanding of the customer and promote continuous communication with real world **examples**.
- Examples describe how the software is **intended to behave**, often illustrating a business rule or requirement.

The main idea is that business analysts, project managers, users or anyone without technical, but with enough business, knowledge can define tests.

EXAMPLE BDD FEATURE

```
Feature: Add new items to the todo list

  As James (the regular user)
  I want to capture the most important things I need to do
  So that I don't leave so many things until the last minute

  Scenario: Adding the first todo item

    Given that James has an empty todo list
     When he adds Buy some milk to his list
     Then his todo list should contain
          Buy some milk
```

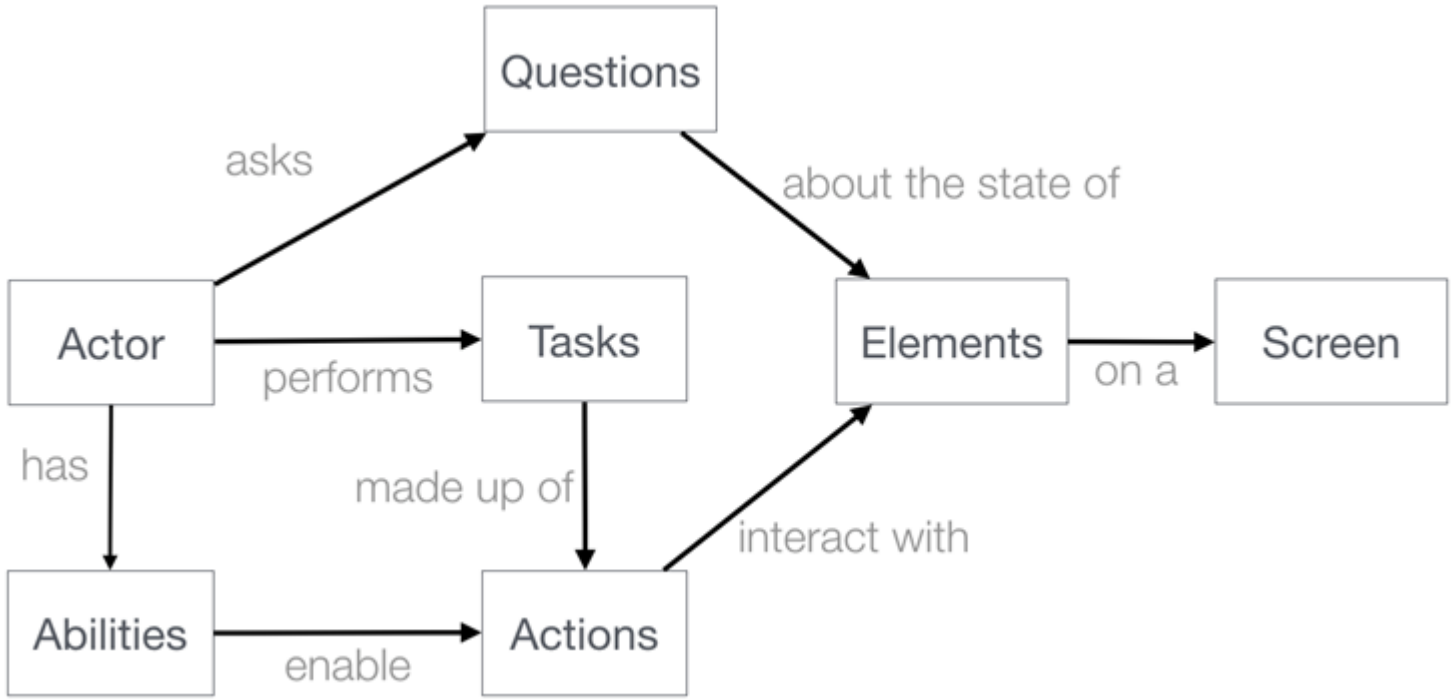
Who is this for? What **Role** do they play?

Why are they here and what outcome do they hope for? What is their **Goal**?

What will they need to do to achieve their goal? What **Tasks** do they need to perform?
How will they complete each task? What will their specific **Interactions** with the system look like?

Question

Screenplay Pattern



The Screenplay pattern is built around **Actors** who use their **Abilities** to perform **Tasks** and ask **Questions** about the state of the system in order to achieve their business goals.

Serenity

```
@Test
public void should_be_able_to_add_a_todo_item() {

    givenThat(james).wasAbleTo(OpenTheApplication.onTheHomePage());

    when(james).attemptsTo(AddATodoItem.called("Buy some milk"));

    then(james).should(seeThat(theDisplayedItems, hasItem("Buy some milk")));
}
```

```
Scenario: Adding the first todo item
    Given that James has an empty todo list
     When he adds Buy some milk to his list
     Then his todo list should contain Buy some milk
```

SERENITY

Layers of abstraction

- **Goals** that represent the high-level business objectives
  - represented by test or scenario name
- **Tasks** that describe the high-level steps the user takes to achieve these goals
  - represented by classes with readable names
- **Actions** that describe how the user interacts with the application to perform each step.

Task

Action

Goal

```
@Test
public void should_be_able_to_add_a_todo_item() {

    givenThat(james).wasAbleTo(OpenTheApplication.onTheHomePage());

    when(james).attemptsTo(AddATodoItem.called("Buy some milk"));

    then(james).should(seeThat(theDisplayedItems, hasItem("Buy some milk")));
}
```

```java
public class AddATodoItem implements Task {

    private final String thingToDo;

    protected AddATodoItem(String thingToDo) { this.thingToDo = thingToDo; }

    public static AddATodoItem called(String thingToDo) {
        return Instrumented.instanceOf(AddATodoItem.class).withProperties(thingTo
    }
}
```

```java
AddATodoItem.called("Buy some milk"))
```

High-level interaction

```java
@Step("{0} adds a todo item called #thingToDo")
public <T extends Actor> void performAs(T actor) {
    actor.attemptsTo(
            Enter.theValue(thingToDo).into(NewTodoForm.NEW_TODO_FIELD),
            Hit.the(RETURN).keyIn(NewTodoForm.NEW_TODO_FIELD)
    );
}
```

Interaction on the browser level

Serenity examples

```
@Test
public void items_left_counter_should_be_decremented_when_an_item_is_completed() {

    givenThat(james).wasAbleTo(OpenTheApplication.onTheHomePage());
    andThat(james).wasAbleTo(AddTodoItems.called("Walk the dog", "Put out the garbage"));

    when(james).attemptsTo(
            Complete.itemCalled("Walk the dog")
    );

    then(james).should(seeThat(TheRemainingItemCount.value(), is(1)));
}
```

```
@Test
public void completed_items_should_not_appear_in_the_active_list() {

    givenThat(james).wasAbleTo(OpenTheApplication.onTheHomePage());
    andThat(james).wasAbleTo(AddTodoItems.called("Walk the dog", "Put out the garbage"));

    when(james).attemptsTo(
            Complete.itemCalled("Walk the dog"),
            FilterItems.byStatus(Active)
    );

    then(james).should(seeThat(theDisplayedItems, not(contains("Walk the dog"))));
}
```

Serenity BDD Report

Test Outcome

Show Shipping Cost For An Item In The Shopping Cart

Shopping Cart (capability)

Story: Add Item To Shopping Cart

In order to make the most appropriate purchase decisions
As a buyer
I want to be able to place items I want to buy in a virtual cart before placing my order

Steps	Screenshot	Outcome	Duration
Given I have searched for 'docking station'		SUCCESS	8.6s
And I have selected a matching item		SUCCESS	3.92s
Select 5th matching item in the list		*SUCCESS*	*2.67s*
When I add it to the cart		SUCCESS	4.22s
Then the shipping cost should be included in the total price		SUCCESS	0.8s
Should see total including shipping for: Listing{name='Wood iPhone Stand Wooden iPhone Docking Station Plum wood iPhone Dock Wooden iPhone holder', price=75.35}		*SUCCESS*	*0.53s*

Screenplay Pattern

Advantages

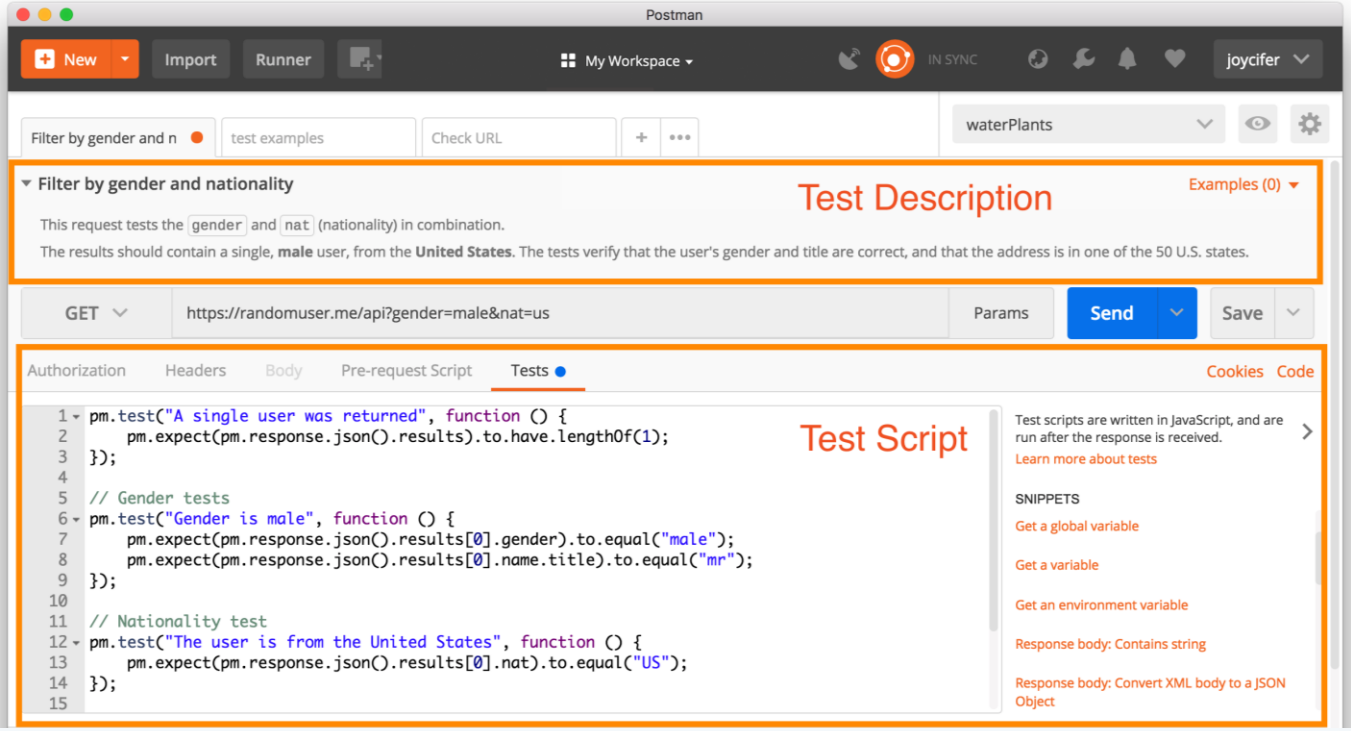
+ Story focused (BDD)
+ Following SOLID principles
+ Acceptance testing
+ Well designed test suites
    + Readability
    + Maintainability
    + Extensibility
+ Composition over inheritance
+ Effective use of layers of abstraction
+ Encourages good testing habits

Disadvantages

- Steep learning curve
- BDD
- Shift in thinking

E2E API TESTING

- Postman JS test collections
  - Verify response content, response status code, content length
  - Measure response time
  - …
- NPM package `newman` for running tests from command line and inside CI environment




```
// example using pm.response.to.have
pm.test("response is ok", function () {
    pm.response.to.have.status(200);
});
```

Test flakiness

My Selenium Tests Aren't Stable!

- Poor test isolation
  - The tests work fine when run alone but fail "randomly" during the build.
- Relying on flaky external services.
  - Using production backends or relying on infrastructure outside of your team's control. All tests fail due to the same underlying cause.
- Timeouts are not long enough
  - Most of the time the tests run fine but fail under load or exceptional circumstances.
- Timeouts are too long
  - Waiting for a page to load by polling for a piece of text, only to have the server throw an exception and give a 500 or 404 error and for the text to never appear.

https://testing.googleblog.com/2009/06/my-selenium-tests-arent-stable.html

Should I Avoid E2E Testing?

End-to-end tests can catch bugs that manifest across your entire system.

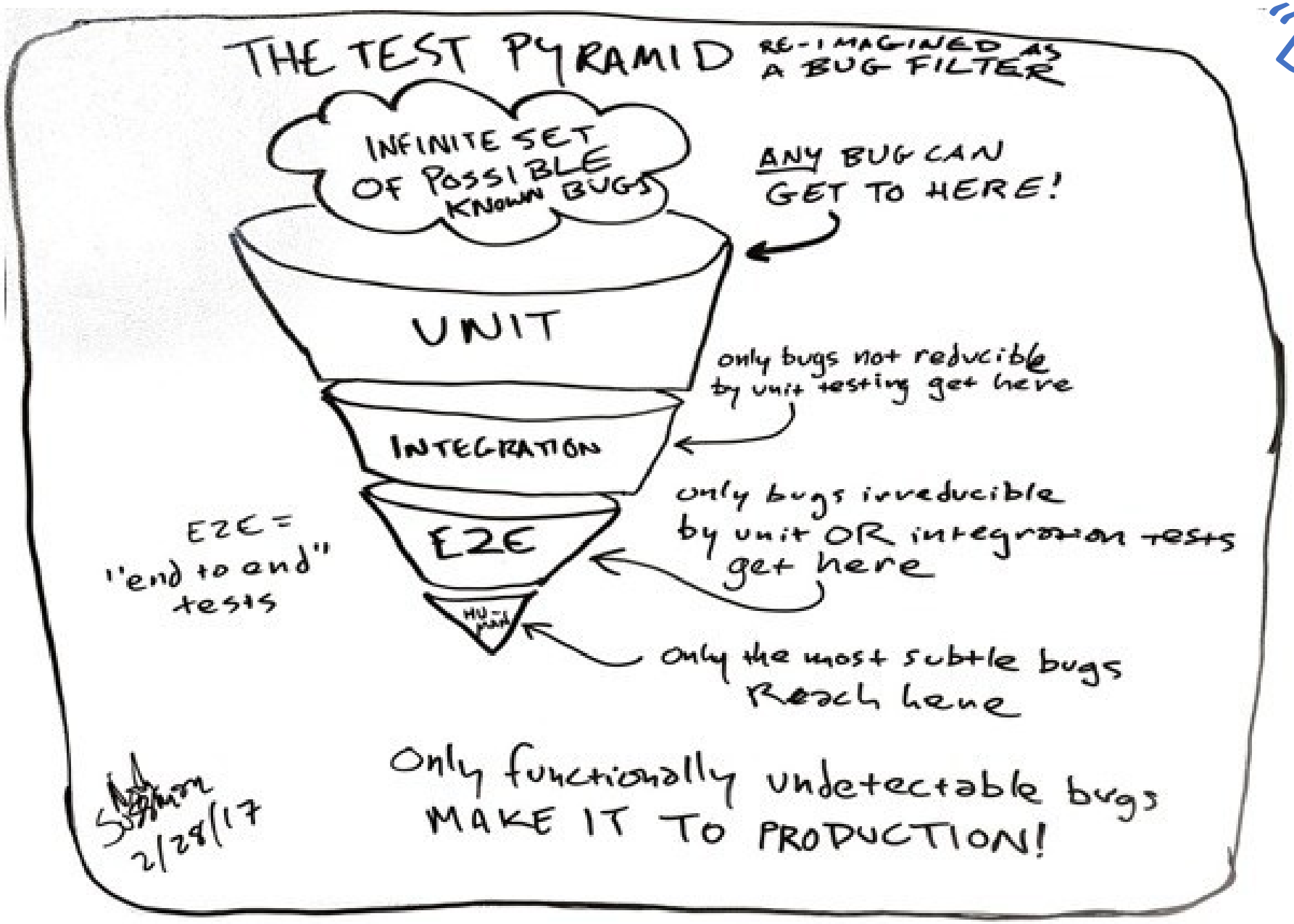
- Tests are flaky and require high maintenance
- Test execution is slow (bugs are discovered later in the report)
- Defects must be located (front-end → line of code)
- Big bugs are hiding small bugs (login process fails → tests fail)

Should I avoid the E2E testing process? No. Even though Google engineer said yes in 2005 (see the link, read the discussion as well).

If you have an UI, you should focus on user. Think about the risks of having a bug in UI.

Build your test strategy from bottom up.

https://testing.googleblog.com/2015/04/just-say-no-to-more-end-to-end-tests.html



"Bug Filter"

Source: https://twitter.com/noahsussman/status/836612175707930625

What Makes a Good End-to-End Test?

In general E2E tests are slower, flakier, and more expensive to maintain than unit or integration tests.

Consider carefully whether an end-to-end test is warranted, and if so, how best to write one.

In order to be cost effective, an E2E test should focus on aspects of the system that cannot be reliably evaluated with smaller tests, such as resource allocation, concurrency issues and API compatibility.

https://testing.googleblog.com/2016/09/testing-on-toilet-what-makes-good-end.html

What Makes a Good End-to-End Test?

- For each important use case, there should be one corresponding end-to-end test. This should include one test for each important class of error.
  - The goal is to keep your total end-to-end count low.
- Allocate time for test maintenance to keep your E2E tests stable in the face of issues like slow and flaky dependencies or minor UI changes.
- Focus on verifying overall system behavior instead of specific implementation details;
  - for example, when testing login behavior, verify that the process succeeds independent of the exact messages or visual layouts, which may change frequently.
- Make your E2E test easy to debug by providing an overview-level log file, documenting common test failure modes, and preserving all relevant system state information (screenshots, database snapshots, etc.).

https://testing.googleblog.com/2016/09/testing-on-toilet-what-makes-good-end.html

Caveats of E2E testing

- System components that are owned by other teams may change unexpectedly and break your tests.
  - This increases overall maintenance cost, but can highlight incompatible changes
- It may be more difficult to make an end-to-end test fully hermetic; leftover test data may alter future tests and/or production systems.
  - Where possible keep your test data ephemeral.
- An E2E test often necessitates multiple test doubles (fakes or stubs) for underlying dependencies.
  - They can, however, have a high maintenance burden as they drift from the real implementations over time.

Sources

https://blog.getgauge.io/are-page-objects-anti-pattern-21b6e337880f (Are page objects antipattern?)

https://www.thucydides.info/#/ (Serenity BDD)

https://serenity-js.org/design/screenplay-pattern.html (Serentity JS)

http://serenity-bdd.info/docs/serenity/#_introducing_the_screenplay_pattern (Serenity Java)

https://johnfergusonsmart.com/getting-started-with-rest-api-testing-with-serenity-and-cucumber/ (Serenity API testing)

https://ideas.riverglide.com/page-objects-refactored-12ec3541990

https://www.infoq.com/articles/Beyond-Page-Objects-Test-Automation-Serenity-Screenplay

http://testerstories.com/2016/06/screenplay-pattern-with-java-part-1/

http://serenity-bdd.info/docs/articles/screenplay-tutorial.html

https://janmolak.com/user-centred-design-how-a-50-year-old-technique-became-the-key-to-scalable-test-automat...

https://learning.getpostman.com/docs/postman/postman_api/continuous_integration/ (Postman)

https://learning.getpostman.com/docs/postman/collection_runs/command_line_integration_with_newman

https://amtoya.com/blogs/test-pyramid-as-a-risk-filter/ (Test Pyramid as a Risk Filter)

THANK YOU

WE ADDED A NEW PERFORMANCE TEST, BUT LEARNED THAT THE TEST ITSELF IS FLAWED.
Dilbert.com DilbertCartoonist@gmail.com
NOW OUR PRODUCT FAILS OUR OWN TESTS AND OUR CUSTOMERS ARE ASKING TO SEE THE TEST RESULTS.
8-11-10 ©2010 Scott Adams, Inc./Dist. by UFS, Inc.
DO I HAVE PERMISSION TO FAKE THE TEST DATA?
I DIDN'T EVEN KNOW DATA CAN BE REAL.